

第一节：什么是硬件

从软件顺序执行到硬件编程模型

软件：PC 推动的顺序执行

软件程序是一串按顺序推进的操作。程序计数器 PC 指向当前指令，完成后指向下一条。

示例程序

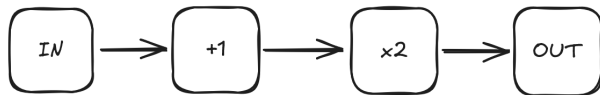
```
x <- input()  
y <- x + 1  
z <- y * 2
```

时刻	PC 指向	当前操作	结果
T0	第 1 行	读取输入	得到 x
T1	第 2 行	计算 $x + 1$	得到 y
T2	第 3 行	计算 $y * 2$	得到 z

关键是「先后」：当前操作完成后，下一条才成为当前操作。

硬件：已经存在的部件网络

硬件不是一串等待执行的语句，而是一组已经存在并相互连接的部件。



输入变化后，信号沿连接关系传播。加法部件和乘法部件同时存在，各自根据当前输入产生输出。

执行模型对比

问题	软件	硬件
什么在推进	PC 和控制流	信号在连接关系中传播
基本单位	指令、语句、函数调用	部件、输入、输出、连接
先后关系	一条指令完成后再转向下一条	独立部件可以同时计算
分支	选择下一条执行路径	选择哪一路结果送入后续部件
资源	同一段代码可以任意复用	并行使用的部件需要同时存在

硬件中的并行不是「同一条指令被更快执行」，而是多个计算关系本来就同时存在。

组合逻辑：硬件里的纯函数

组合逻辑是没有记忆的输入输出函数：

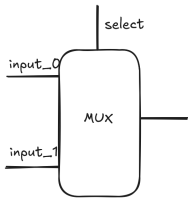
$$\text{output} = f(\text{input})$$

- 同一组输入永远得到同一组输出。
- 输出不依赖过去，只依赖当前输入。
- 计算关系持续存在，不是「被调用后才计算」。

线缆负责传递当前值，组合逻辑负责把当前值变成另一个当前值。

例子：2 选 1 MUX

MUX 根据 select 选择一路输入送到输出。



$$\text{output} = (!\text{select} \& \text{input}_0) \mid (\text{select} \& \text{input}_1)$$

select	input_0	input_1	output
0	0	x	0
0	1	x	1
1	x	0	0
1	x	1	1

软件 if: 选择控制流

软件的分支只执行满足条件的那一条路径。

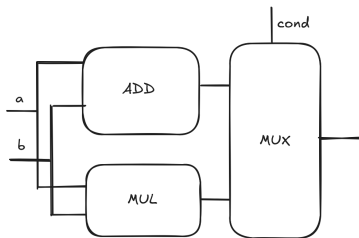
软件写法

```
if cond:  
    result = a + b  
else:  
    result = a * b
```

- cond 决定下一段被执行的代码。
- 没被选中的路径不会在这一轮执行。

硬件 if: 两条路径都提前存在

硬件的对应结构是：加法器和乘法器同时存在，最后由 MUX 选择结果。



cond 控制的是「选哪个结果」，不是「现在才制造一个加法器」或「跳转到乘法代码」。

硬件规模由设计时结构决定

一个运行中的信号可以控制已有部件，但不能凭空制造一个此前不存在的部件，也不能在运行中改变连接数量。

组合逻辑的边界：循环

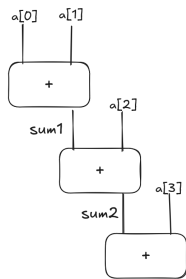
软件循环依赖重复使用同一段逻辑：

```
for i = 0 ... n - 1:  
    sum = sum + a[i]
```

组合逻辑下，一次信号传播只能穿过已经画好的有限连接，不能根据运行时的 n 临时多走几遍同一块电路。

固定次数：展开成硬件

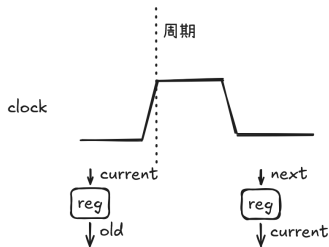
如果循环次数固定为 4，组合逻辑能做的是把加法链展开成 4 个加法器。



软件循环第几次	组合逻辑里对应的硬件
$i = 0$	第 1 个加法器
$i = 1$	第 2 个加法器
$i = 2$	第 3 个加法器
$i = 3$	第 4 个加法器

时钟与寄存器

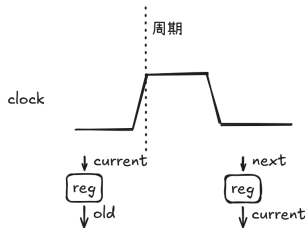
组合逻辑只能根据当前输入算当前输出，不会记住任何东西。要让本周期结果留到下一周期，硬件需要寄存器。



时钟是周期性跳变的信号，上升沿是所有寄存器统一采样更新的节拍点。

寄存器行为

- 两个时钟边沿之间，输出 `current_value` 保持不变。
- 边沿到来时，采样输入 `next_value`。
- 采样后，输出更新为刚采样的值。



结论

没有寄存器，硬件就没有「上一周期的值」。

时序逻辑 = 组合逻辑 + 寄存器

两者连接后的通用模型：

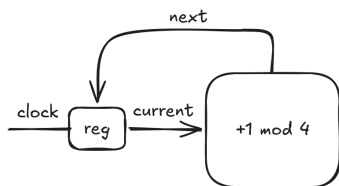
$$S_{t+1} = f(S_t, I_t)$$

- 组合逻辑负责计算下一状态。
- 寄存器负责在周期边界保存下一状态。
- 计数器、循环式硬件、CPU 都符合这个模型。

例子：2 位计数器的周期行为

组合逻辑计算：

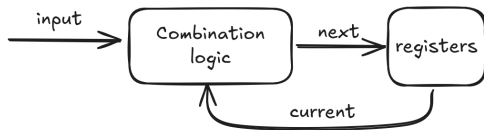
$$\text{next} = (\text{current} + 1) \bmod 4$$



周期	周期开始	计算	周期结束
0	00	+1	01
1	01	+1	10
2	10	+1	11
3	11	+1	00

当前状态经组合计算得到下一状态，寄存器在周期边界保存它。

小结



- 线缆：传递当前值，无记忆。
- 组合逻辑：无记忆的函数，输入变、输出跟着变。
- 寄存器 + 时钟：硬件记忆的唯一来源，决定「何时」采纳新状态。
- 并行 = 多个部件同时存在；分支 = MUX 选择。